SBDL text cheatsheet

Human-readable encoding of SBDL (Sight Bundle): schemas in `.sgs` files, data in `.sbdt` files, lossless conversion to and from binary `.sbdl`. Full specification: sbdl-text.md, binary encoding: sbdl-binary.md.

Lexical essentials

Files	`.sgs` schema · `.sbdt` text data · `.sbdl` binary (data or compiled schema)
Encoding	UTF-8 **without BOM**, `LF` or `CR LF` (a lone `CR` is an error)
Comments	`// line`, `/* block */` (not nested)
Separators	comma **or** line break between items; trailing comma allowed; `[1 2]` is an error
Identifiers	`[A-Za-z_][A-Za-z0-9_]*`, qualified names `a.b.c`
Names	ordinal comparison (case-sensitive)
Same line	a type prefix and its `{` / `(`, an explicit type and its value
Directives	`@name …` at the start of a line, before the content

Reserved words (never identifiers): `null true false inf nan`

Type keywords (never type names): `bool u8 u16 u32 u64 s8 s16 s32 s64 f32 f64 string bytes list map ref any`

Contextual keywords (still valid field names): `enum struct abstract type`

Literals

Literal	Examples
Integers	`42`, `-7`, `007`, `1_000_000`, `0x2A`, `0xFF_FF`, `0b1010`
Integer suffix	`255u8`, `-3s16`, `0xFFu32` (`u8 … u64`, `s8 … s64`)
Floats	`3.14`, `-1e-3`, `2E10`, `inf`, `-inf`, `nan` — digits on both sides of `.`
Float suffix	`1.5f32`, `2f32`, `nanf64`
Booleans, null	`true`, `false`, `null`
Strings	`"tab\there"` — escapes `\" \\ \n \r \t \0 \u{1F600}`
Multi-line	`"""` + line break … closing `"""` alone on its line (its indentation is removed)
Base64	`b64"SGVsbG8="` — padding mandatory, whitespace ignored, only for `bytes`

```
text: """
    First line
      indented line
    """                    // "First line\n  indented line"
```

Types

Type	Notes
`bool`	`true` / `false`
`u8 u16 u32 u64` · `s8 … s64`	varint encoded, raw with `@fixed` (16–64 bits)
`f32 f64`	IEEE 754, bit-exact round trip
`string`	UTF-8
`bytes`	shorthand of `list<u8>`, Base64 in text
`list<T>`	ordered
`map<K, V>`	`K`: `bool`, integer, `string` or enum; insertion order kept
`ref<T>`	struct `T`: in place, internal (`*a`) or external (`<dep>`)
`any`	value carrying its type
`Name`	enum, struct or type alias
`T?`	nullable — not on `any`, never `T??`

Schema (`.sgs`)

```
@sgs 1                          // first line, mandatory
@id game                        // schema name, mandatory
@version 3
@meta author = "Arthur"
@import assets 2                // types usable as assets.X
@import tex = assets.textures 1  // local alias: tex.X

enum Class : u8 { Warrior, Mage = 5, Rogue }   // 0, 5, 6

type Color = u32                // alias: same encoding
type Palette = list<Color>

struct Vec3 @positional { x: f32, y: f32, z: f32 }

abstract struct Character {
  name:  string
  level: u8 = 1                 // default: not written
}

struct Warrior : Character { rage: u16 }
struct Mage : Character { mana: u32 }

struct Player {
  party:  list<ref<Character>> // polymorphic
  tint:   Color = 0xFFFFFFFF
  hash:   u64 @fixed           // raw 8 bytes
  xp:     s32 @alias(exp, experience)  // former names
  leader: ref<Character>?
  pos:    Vec3
  icon:   ref<assets.Texture>
  avatar: bytes?
  extra:  any
}
```

Schema rules

- Types may be used before their declaration; type names are unique in a schema.
- Defaults only on non-nullable `bool`, numbers, `string`, enums. `T? = v` is invalid: `null` is the default of `T?`.
- `@fixed` on `u16 … s64` fields (possibly through an alias), not on aliases themselves.
- `@alias(…)` gives former field names, used when reading data written with an older schema.
- `type A = T`: `T` not nullable, no cycles; an alias is never a `ref<…>` target nor a base.
- `@positional`: struct not abstract, only non-nullable scalar or enum fields (inherited ones included).
- Single inheritance, no cycle; `abstract` structs are never instantiated.
- Unqualified names: current schema, then direct imports (ambiguous → qualify).

Data document (`.sbdt`)

```
@sbdl 1                             // very first line
@id "level-01"
@version 3
@meta author = "Arthur"
@schema game 3                      // or inline: @schema { … }
@dep tex = assets.textures 2        // alias = id version

Player {                            // root: type prefix mandatory
  party: [
    Warrior { name: "Conan", rage: 12 }
    &hero Mage { name: "Merlin", level: 7, mana: 300 }
  ]
  hash: 0x9F3A_22C1_0000_0001
  xp: 1200
  leader: *hero                     // internal reference
  pos: (1.5, 0.0, -3.0)             // positional struct
  icon: <tex#hero>                  // export of a dependency
  extra: 42u8                       // any: typed literal
}

@export captain = *hero             // after the root
```

Directives

Directive	Count	Meaning
`@sbdl 1`	1, first line	text format version
`@id name` / `@id "any text"`	≤ 1	file id (indexed by search paths)
`@version n`	≤ 1	content version (default 0)
`@meta key = "value"`	any	metadata, unique keys

Directive	Count	Meaning
`@schema name version [0xHASH]`	1+	referenced schemas (hash checked if given)
`@schema { … }`	1+	inline schemas (no `@sgs` inside), not mixed with references
none		no `@schema`: schemas given by `SerializationOptions.Schemas`
`@dep alias = id version`	any	dependency, `alias` local to the file
`@export name = value`	any, after root	`*anchor` or prefixed struct

Values by expected type

Expected type	Accepted syntax
integers	integer literal in range; suffix = the type, if any
`f32` `f64`	float or decimal integer (`2` → `2.0`)
`string`	`"…"` or `"""…"""`
enum `E`	`Mage`, `Class.Mage`, `game.Class.Mage`
`bytes`	`b64"…"` or `[1, 2, 3]`
`list<T>`	`[a, b, c]`
`map<K, V>`	`{ "str": 1.5, "dex": 8 }` — keys: `"…"`, integers, `true` / `false`, enum names
struct `T`	`{ field: value }` or `(a, b, c)`, optional `T` prefix
`ref<T>`	struct value, `&a` + struct, `*a`, `<dep>` or `<dep#export>`
`T?`	`null` or a value of `T`
alias	syntax of its target

Structs & type prefixes

```
inventory: [
  { name: "Potion", count: 3 }
  { name: "Torch" }            // count: default (1)
]
party: [Warrior { name: "Conan" }]   // prefix: real type
```

- Fields in any order, once each; unknown field → error with suggestion (`did you mean 'level'?`).
- Missing field: `null` if nullable, schema default otherwise, else error. No field aliases in text.
- Prefix **mandatory**: root, exports, `any`, real type ≠ declared type, abstract declared type.
- Prefix **optional** otherwise: the declared type or a derived one. An alias of a struct is accepted.

Positional values

```
pos:  (1.5, 0, -3)            // x: 1.5, y: 0.0, z: -3.0
path: [(0, 0, 0), (1, 0, 0)]
size: Rect (0, 0, 1920, 1080)  // with prefix
```

- Every field, in field sequence order (base fields first), defaults included.
- Any struct with only non-nullable scalar / enum fields accepts it; writers use it only for `@positional` structs.
- Always a struct, never a map.

References

```
@dep tex = assets.textures 2

textures: [
  &1 { path: "local.png" }     // in place, anchored
  *1                           // internal: same object
  <tex>                        // root of the dependency
  <tex#hero>                   // export "hero"
  <tex#"any name">             // quoted export name
]
```

- One `ref<T>` position accepts the three forms, value by value.
- Anchors `&name` / `&12`: before a struct at a `ref<T>`, export or `any` position only.
- `*name` may come before its anchor (cycles allowed); it must resolve.
- The target type must be `T` or derive from it.

Values of type `any`

Syntax	Type
`null` / `true`	Null / `bool`
`42` / `42u16`	`s64` / suffix type
`1.5` / `1.5f32`	`f64` / suffix type
`"text"` / `b64"…"`	`string` / `bytes`
`Class.Mage`	enum (qualified)
`Item { … }` / `Vec3 (1, 2, 3)`	struct (prefix mandatory)
`[ … ]`	`list<any>`
`{ "k": … }`	`map<string, any>`
`&a Item { … }`, `*a`	`ref<Item>`
`Type value`	explicit type: `list<s32> [1, 2]`, `map<Class, u8> { Mage: 1 }`, `ref<Texture> <tex>`, `Color 255`

Aliases are resolved in `any`: `Color 255` is stored as `255u32`.

Canonical output

- `LF`, 2-space indent, final line break; directives in table order, then a blank line and the root.
- Containers on one line if they fit in 80 columns: `[1, 2, 3]`, `{ name: "Potion", count: 3 }`; otherwise one item per line, without commas.
- Fields in schema order; nulls and defaults omitted (option `WriteDefaults`).
- Prefixes only where mandatory; enum values unqualified outside `any`.
- Integers in decimal; floats: shortest round-trip form (`1.0`, `1e21`, `1.5e-7`).
- `bytes` → Base64; `@positional` structs → `( … )`.
- Anchors `&1`, `&2` … only on shared or exported objects; dependency alias = last segment of the id.

Common errors

Message	Cause
`Unknown field 'levle' in 'game.Player' (did you mean 'level'?)`	typo, or field of another type
`Missing required field 'name' in …`	field without default nor `?`
`Expected ',' or a line break before …`	`[1 2]`, `{ a: 1 b: 2 }`
`A type prefix is mandatory here`	root / export without `Type { … }`
`A type prefix is mandatory: '…' is abstract`	`{ … }` for an abstract declared type
`Enum value 'Mage' must be qualified with its type in 'any'`	write `Class.Mage`
`This value requires an explicit type in 'any'`	`<tex>` in `any` → `ref<T> <tex>`
`Struct '…' has 3 fields, 2 values are given`	positional value incomplete
`Unresolved reference '*a'`	no `&a` anchor in the document
`Undeclared dependency alias 'tex'`	missing `@dep tex = …`
`An anchor is only allowed at a reference position`	`&a` on a plain struct field
`Base64 literals are only valid for list<u8>`	`b64"…"` elsewhere

C# API (`Sight.Serialization`)

Read & write

```
// Typed (schema from the C# types, inline)
string text = TextSerializer.Write(player);
Player p = TextSerializer.Read<Player>(text);

// Untyped
DataDocument doc = TextSerializer.Read(text, options);
TextSerializer.Write(stream, doc, options);

// Text <-> binary
var data = TextSerializer.Read(text, options);
byte[] bin = BinarySerializer.Write(data, options);
data = BinarySerializer.Read(bin, options);

// Schemas
Schema s = Schema.Parse(sgsText, options);
Schema t = Schema.FromType<Player>();
string sgs = s.ToText();
Schema.Compile(sgsStream, sbdlStream);  // to .sbdl
```

All operations have `Async` variants (`ReadAsync` , `WriteAsync` , `ParseAsync` ...) with a `CancellationToken` .

Options

`SerializationOptions`	
`Schemas`	schemas in memory (mandatory without `@schema`)
`SearchPaths`	folders indexed for `.sbdl` , `.sbdt` , `.sgs` (first wins)
`Dependencies.Add(id, version[, value | Func<Stream>])`	dependencies in memory or declared
`Resolvers`	custom `IResourceResolver` (archives, network...)
`AllowNewerDependencies`	accept a newer dependency version
`Id` , `Version` , `Metadata` , `Exports`	header and exports when writing
`SchemaMode`	`Inline` (default), `Reference` , `None`
`Indent` / `MaxInlineWidth` / `WriteDefaults`	text layout: `2` / `80` / `false`
`Limits`	depth, collection, string and object limits
`OnDiagnostic`	warnings (shadowed files...)

Errors are `SerializationException` with `Line` and `Column` for text input.

Mapping attributes

Attribute	Effect
`[SerializationType("game.Player")]`	type identity; `IsReference` , `IsPositional`
`[SerializationDerivedType(typeof(Mage))]`	derived types known through the base
`[assembly: SerializationSchema("game", 3)]`	schema version
`[SerializationField("lvl")]`	field name (camelCase of the member by default)
`[SerializationAlias("xp", "exp")]`	former names (`@alias`)
`[SerializationIgnore]`	not serialized
`[SerializationFixed]`	`@fixed`
`[SerializationReference]`	member as `ref<T>` (identity kept)
`[DefaultValue(1)]`	schema default, not written
`[SerializationConverter(typeof(X), args…)]`	surrogate conversion (member, type or assembly)

CLR types

C#	Schema
`bool` , `byte` … `ulong` , `sbyte` … `long` , `float` , `double` , `string`	scalars
`byte[]` , `ReadOnlyMemory<byte>` , `Memory<byte>` , `ArraySegment<byte>`	`bytes`
arrays, `List<T>` , `IList<T>` , `HashSet<T>` …	`list<T>`
`Dictionary<K, V>` , `IReadOnlyDictionary<K, V>` …	`map<K, V>`
enum, class / struct	enum, struct
`Link<T>`	`ref<T>` (local or external)
`object`	`any`
`T?` , nullable annotations	`T?`

Links (`ref<T>`)

```
public List<Link<Texture>> Links { get; set; }

atlas.Links.Add(texture);      // local (implicit)
atlas.Links.Add(texture);      // same object: shared
atlas.Links.Add(new Link<Texture>("assets.lib", "hero"));

Link<Texture> l = atlas.Links[2];
l.IsExternal; l.DependencyId; l.Key;
Texture t = l.Resolve();       // loaded on demand, cached
```

- `Link<T>` keeps external references unresolved; a plain `T` member at a `ref<T>` position resolves them while reading.
- Writing an external link needs its dependency version (`Link` constructor or `Dependencies`).

Converters

```
// Named surrogate: type Color = u32
[SerializationType("game.Color")]
public sealed class ColorConverter
    : SerializationConverter<Color, uint>
{
    public override uint ToSurrogate(Color c) => c.Rgba;
    public override Color FromSurrogate(uint s) => new(s);
}

// Every Color
[SerializationConverter(typeof(ColorConverter))]
public readonly struct Color { … }

// Third-party type
[assembly: SerializationConverter(typeof(Vec3Conv))]

// One member, with a constructor argument: s32
[SerializationConverter(typeof(FixedPoint), 8)]
public float Speed { get; set; }
```

- The schema comes from the surrogate type only; arguments configure the conversion.
- Converters never receive `null` ; in `object` members the surrogate is stored.